

Practical 6th

Aim: A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint)' The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user.

Write a program to :

- Calculate current need of resources by each process Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

Code:

```
GNU nano 8.7.1 bankers.cpp
#include <iostream>
using namespace std;

int main() {
    int n, m;

    cout << "Enter number of processes: ";
    cin >> n;

    cout << "Enter number of resources: ";
    cin >> m;

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    cout << "Enter Allocation Matrix:\n";
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            cin >> alloc[i][j];

    cout << "Enter Max Matrix:\n";
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            cin >> max[i][j];

    cout << "Enter Available Resources:\n";
    for(int i=0; i<m; i++)
        cin >> avail[i];

    // Need matrix calculate
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    bool finish[n] = {false};
    int safeSeq[n];
    int work[m];

    for(int i=0; i<m; i++)
        work[i] = avail[i];

    int count = 0;

    while(count < n) {
        bool found = false;

        for(int i=0; i<n; i++) {
            if(!finish[i]) {
                bool possible = true;

                for(int j=0; j<m; j++) {
                    if(need[i][j] > work[j]) {
                        possible = false;
                        break;
                    }
                }

                if(possible) {
                    for(int j=0; j<m; j++)
                        work[j] -= need[i][j];

                    finish[i] = true;
                    safeSeq[count] = i;
                    count++;
                }
            }
        }
    }

    cout << "Safe Sequence: ";
    for(int i=0; i<n; i++)
        cout << safeSeq[i] << " ";
    cout << endl;

    return 0;
}
```

[Read 82 lines]

AG Help	AO Write Out	AF Where Is	AK Cut	AT Execute	AC Location	M-U Undo
AX Exit	AR Read File	AN Replace	AU Paste	AJ Justify	AV Go To Line	M-E Redo

```

        for(int j=0;j<m;j++) {
            if(need[i][j] > work[j]) {
                possible = false;
                break;
            }
        }

        if(possible) {
            for(int j=0;j<m;j++)
                work[j] += alloc[i][j];

            safeSeq[count++] = i;
            finish[i] = true;
            found = true;
        }
    }
}

if(!found) {
    cout << "System is NOT in safe state";
    return 0;
}

cout << "System is in SAFE state\n";
cout << "Safe sequence: ";

for(int i=0;i<n;i++)
    cout << "P" << safeSeq[i] << " ";

return 0;
}

```

Help	Write Out	Where Is	Cut	Execute	Location	Undo
Exit	Read File	Replace	Paste	Justify	Go To Line	Redo

Output:

```

priti@LAPTOP-US9V80AL UCRT64 ~
$ ./bankers
Enter number of processes: 5
Enter number of resources: 4
Enter Allocation Matrix:
0 0 1 4
0 6 3 2
0 0 1 2
1 0 0 0
1 3 5 4
Enter Max Matrix:
0 6 5 6
0 6 5 2
0 0 1 2
1 7 5 0
2 3 5 6
Enter Available Resources:
1 6 2 0
System is in SAFE state
Safe sequence: P1 P2 P3 P4 P0
priti@LAPTOP-US9V80AL UCRT64 ~
$ |

```